

**CONSOLATRIX COLLEGE OF
TOLEDO CITY, INC.**

COLLEGE OF COMPUTER STUDIES

Software Requirements Specifications

for

Lokal: A Web Application for Online Hiring and Applicant Management for
CJTECH Computer Trading

Change History

Table of Contents

Change History	2
Table of Contents	3
1. Introduction	4
1.1. Purpose	4
1.2. Scope	4
1.3. Definitions, Acronyms and Abbreviations	5
1.4. References	5
2. Overall Description	7
2.1. Product perspective	7
2.2. User characteristics	9
2.4. Constraints	10
2.5. Assumptions and dependencies	10
3. Specific Requirements	11
3.1. External interface requirements	11
3.1.1. <i>Hardware interfaces</i>	11
3.1.2. <i>Software interfaces</i>	11
3.1.3. <i>Communications interfaces</i>	12
3.2. Functional requirements	13
<i>Module 1</i>	13
<i>Module 2</i>	20
3.4. Non-functional requirements	29
<i>Performance</i>	29
<i>Security</i>	29
<i>Reliability</i>	30

1. Introduction

1.1. Purpose

The purpose of this Software Requirements Specification (SRS) is to give a detailed overview of **Lokal**, a web application for online hiring and applicant management for CJTECH Computer Trading. This document outlines the functional and non-functional requirements needed for the successful development, testing, and deployment of the system. It acts as a reference for developers, testers, project advisers, and stakeholders to ensure everyone understands the system's objectives, scope, and expected features.

1.2. Scope

The **Lokal** is a web-based application that enables **CJTECH Computer Trading** to modernize its recruitment process, manage job postings, and track applicant activity online. It allows for automated profile matching through the Google Gemini API, which contributes to **AI-powered evaluation** of job seekers based on their skills.

Key Features of the Platform Include:

- **Job Browsing and Search:** Applicants can search available vacancies by title or description.
- **Online Application Submission:** Applicants submit applications electronically with data pulled straight from their profiles.
- **Profile Management:** Users can update their name, contact info, location, skills, bio, and resume link at any time.
- **AI-Based Skill Matching:** The system compares applicant skills against job requirements using Google Gemini AI, and falls back to a basic keyword matcher if the AI is unavailable.
- **Re-apply for Rejected Applications:** Applicants whose applications were rejected can re-apply with a single click.
- **Hire / Reject Decisions:** Administrators can mark applicants as hired or rejected directly from the applicant detail view.
- **Job Posting Management:** Administrators can create, edit, and delete job openings with an icon picker for visual flair.
- **Applicant Ranking:** Applicants are ranked by their AI match score so admins see the best candidates first.
- **Application Tracking:** Applicants can see the status of all their submissions in one place.

1.3. Definitions, Acronyms and Abbreviations

Terms	Definition
Lokal	The web-based online hiring and applicant management system developed for CJTECH Computer Trading..
User	Any person using the system, including applicants and administrators.
Applicant	An individual seeking employment who applies for job vacancies through the system.
Admin	A user with administrative privileges to manage jobs, applicants, and system records.
Job posting	An advertisement containing the position title, description, requirements, salary, and location.
Application	A submission made by an applicant for a specific job vacancy.
AI-Based Matching	A feature that evaluates applicant skills against job requirements using the Google Gemini API.

1.4. References

1. IEEE Standard for Software Requirements Specifications

Title: IEEE Std 830-1998

Date: 1998

Publishing Organization: Institute of Electrical and Electronics Engineers (IEEE)

Source: IEEE Xplore Digital Library

2. React Documentation (v18)

Title: React Documentation

Date: Ongoing

Publishing Organization: Meta Open Source

Source: <https://react.dev>

3. Vite Documentation

Title: Vite Frontend Tooling Documentation

Date: Ongoing

Publishing Organization: Evan You

Source: <https://vitejs.dev>

4. Supabase Documentation

Title: Supabase Documentation

Date: Ongoing

Publishing Organization: Supabase Inc.

Source: <https://supabase.com/docs>

5. Google Gemini API Documentation

Title: Google Gemini API Documentation

Date: Ongoing

Publishing Organization: Google

Source: <https://ai.google.dev>

6. PostgreSQL Documentation

Title: PostgreSQL Database Documentation

Date: Ongoing

Publishing Organization: PostgreSQL Global Development Group

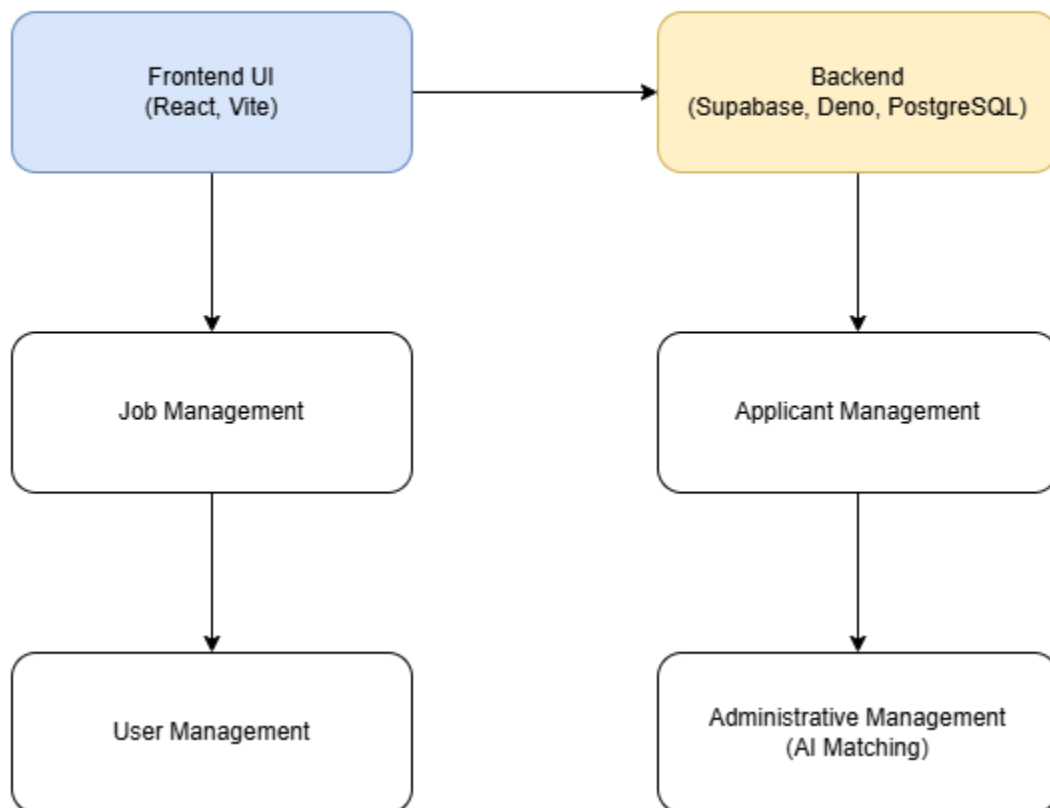
Source: <https://www.postgresql.org/docs>

2. Overall Description

2.1. Product perspective

The **Lokal: An Online Hiring and Applicant Management System for CJTECH Computer Trading** is a standalone web application built with **React** and **Vite** on the frontend, and **Supabase** (which provides **PostgreSQL**, authentication, and storage) on the backend. The AI matching feature uses the Google Gemini API with a built-in fallback to basic substring matching so the app still works even if the API key expires or the network is down. The system does not depend on any external recruitment platforms. It is self-contained: user accounts, job data, applications, and notifications all live inside the Supabase PostgreSQL database. The design is modular enough that future versions could hook into third-party email services or external job boards without rewriting the core.

Lokal - Block Diagram



Modular Decomposition

Module 1: User Management

Transaction 1.1: User Registration (Signup)

Transaction 1.2: User Authentication (Login / Logout)

Transaction 1.3: Profile Management (Edit name, phone, location, skills, bio, resume link, avatar)

Module 2: Job Management

Transaction 2.1: Browse and Search Jobs

Transaction 2.2: Create / Edit / Delete Job Postings (Admin)

Transaction 2.3: View Job Details

Module 3: Application Management

Transaction 3.1: Submit Application (data sourced from profile)

Transaction 3.2: Track Application Status

Transaction 3.3: Re-apply for Rejected Applications

Module 4: AI-Based Applicant Matching

Transaction 4.1: AI Skill Analysis via Google Gemini API

Transaction 4.2: Applicant Ranking by Match Score

Transaction 4.3: Basic Keyword Matching Fallback

Module 5: Admin Dashboard

Transaction 5.1: Dashboard Overview (job, applicant, and application counts)

Transaction 5.2: Manage Applicants (view ranked list, hire / reject)

Transaction 5.3: User Management (view, approve, suspend, delete user)

2.2. User characteristics

The **Lokal** system is designed for two types of users, each with specific roles and privileges:

1. Applicant:

- **Role:** The primary user looking for work.
- **Privileges:**
 - Create and manage their own profile (name, phone, location, skills, bio, resume).
 - Upload a profile avatar.
 - Browse and search job openings.
 - Submit applications to jobs they are interested in.
 - Track the status of all their submitted applications.
 - Re-apply to jobs where their application was rejected.
 - Receive notifications about application updates and hiring decisions.

2. Administrator

- **Role:** The recruitment staff or system administrator.
- **Privileges:**
 - Create, edit, and delete job postings with an icon picker.
 - View a ranked list of applicants for any job, ordered by AI match score.
 - See a detailed comparison of basic keyword matching vs. AI-powered matching.
 - Mark applicants as hired or rejected.
 - Manage user accounts (view, approve, suspend, delete).
 - View dashboard statistics (total jobs, applicants, applications).

2.4. Constraints

Several constraints limit the development and operational flexibility of the **Lokal** system:

1. **Regulatory Policies:**
 - Must comply with the Philippine **Data Privacy Act of 2012 (RA 10173)** for protecting applicant personal information.
2. **Hardware Limitations:**
 - Runs on **standard web infrastructure**. No specialized hardware is needed, though server load increases with concurrent users.
3. **Browser Support:**
 - The system targets **modern browsers** (Chrome, Firefox, Edge, Safari). Older browsers may not support all CSS and JavaScript features used.
4. **AI API Dependency:**
 - The AI matching feature depends on the **Google Gemini API**. If the API key expires or the service is unreachable, the system falls back to **basic keyword matching**.
5. **Internet Requirement:**
 - A **stable internet connection** is required for both applicants and administrators to access the platform.
6. **Security:**
 - Passwords are hashed by **Supabase Auth**. Applicant data is protected through **Row-Level Security** policies on the database.

2.5. Assumptions and dependencies

Several assumptions were made during the design and development of the **Lokal**, which, if changed, could impact the system requirements:

1. **Assumption 1:** Users will access the platform through **modern web browsers** with **JavaScript enabled**.
2. **Assumption 2:** Applicants will provide **accurate and complete information** in their profiles for the **AI matching** to work effectively.
3. **Assumption 3:** Administrators will provide **accurate job descriptions and requirements** to ensure meaningful match scores.
4. **Assumption 4:** The system assumes a **stable internet connection** is available for both applicants and administrators.
5. **Dependency 1:** The **Google Gemini API** will remain available under the **free-tier quota** during the project demonstration period.
6. **Dependency 2:** The **Supabase free tier** will provide sufficient **storage and request capacity** for the scale of this capstone project.

3. Specific Requirements

3.1. External interface requirements

3.1.1. Hardware interfaces

Lokal is designed to run on standard web infrastructure with minimal hardware demands. The system is hosted on Supabase's cloud servers, so no on-premise hardware is required. End users only need a device with a modern web browser and an internet connection.

- **Supported devices:**
 - **Desktop and laptop computers:** Recommended screen resolution of 1024px or wider for the full dashboard experience.
 - **Mobile devices** (smartphones and tablets): Responsive interface allows users to access the system on smaller screens.

3.1.2. Software interfaces

The system integrates several software components to function properly. The required software includes:

- **Frontend Framework:**
 - React 18 with Vite is used to build the user interface.
 - Functional components with React Hooks are used throughout the application.
- **Backend / Database::**
 - Supabase provides PostgreSQL, user authentication, file storage, and Row-Level Security (RLS).
 - User information and uploaded files are securely stored in the database.
- **AI Integration:**
 - Google Gemini API (gemini-2.0-flash-lite) is used for AI-powered skill matching.
 - The system communicates with the API through a utility module that handles requests and errors.
- **Styling Framework:**
 - CSS and React component styles are used to create a responsive and user-friendly interface.

3.1.3. Communications interfaces

The **Lokal** system communicates securely between users and system components using the following technologies:

- **Network Protocols:**
 - The system uses HTTPS for secure communication between users, Supabase, and the Google Gemini API.
 - All data transmitted between the client and server is encrypted.
- **API Communication:**
 - The application communicates with Supabase APIs for authentication, database operations, and file storage.
 - The Google Gemini REST API is used for AI-based skill matching and recommendation features.
- **Future Communication Support:**
 - Although WebSockets are not currently implemented, Supabase provides real-time capabilities that may be enabled in future versions.

3.2. Functional requirements

Module 1 – User Management

1.1 Transaction Name: User Registration

- Use Case Diagram

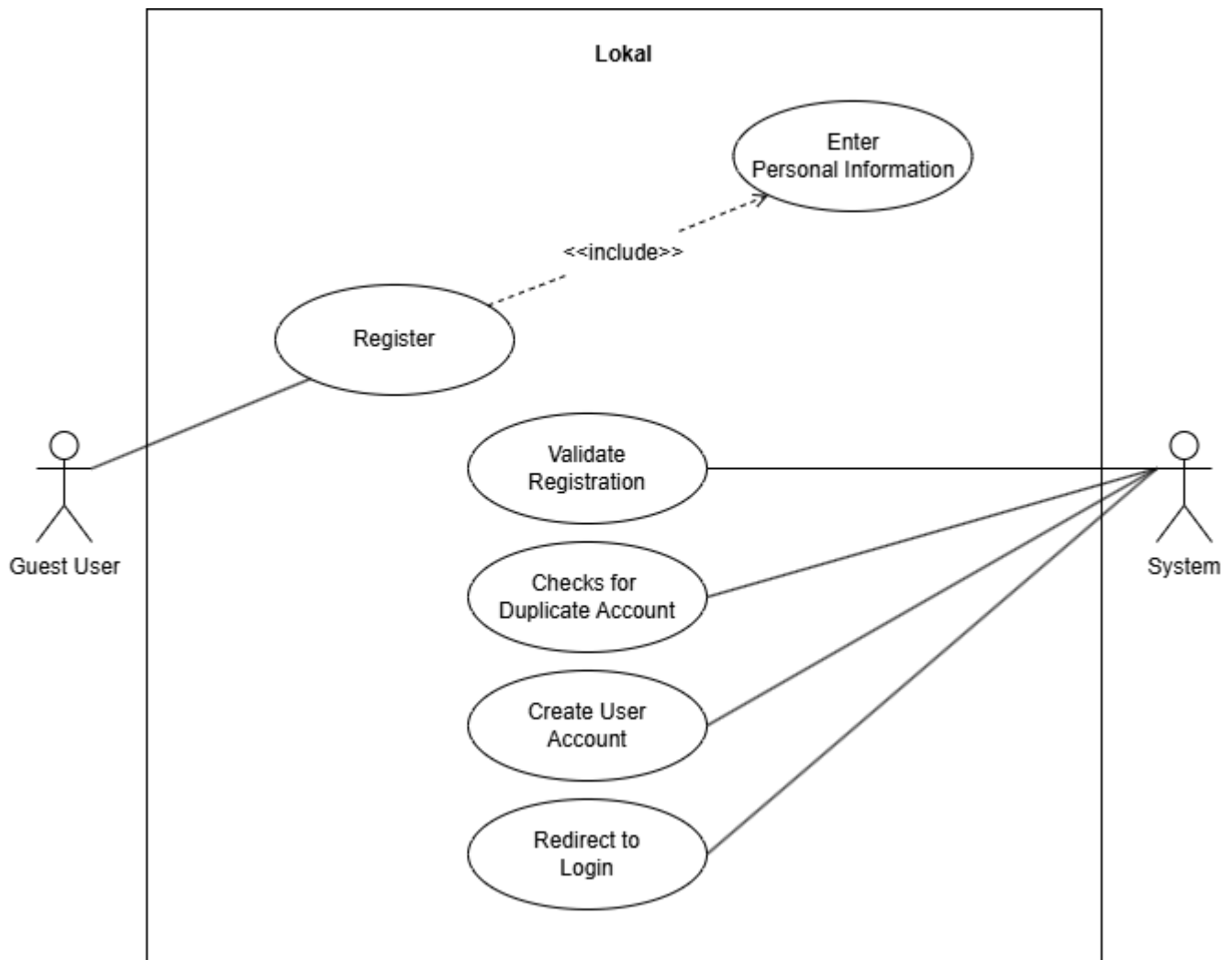


Figure 1. Use Case Diagram for User Registration

- *Use Case Description*

The guest user provides their full name, email address, password, and optional phone number to create an account. Upon successful registration, the system creates the user's account and requires them to sign in.

- **Preconditions:** The user must not already have an account using the same email, and all required fields must be completed.
- **Postconditions:** The account is successfully created, and the user is redirected to the Login page.

Main Success Scenario:

1. The guest user accesses the registration page.
2. The user enters required information.
3. The system validates the entered details.
4. The system creates the user account.
5. The user is redirected to the Login page.

- *Activity Diagram*

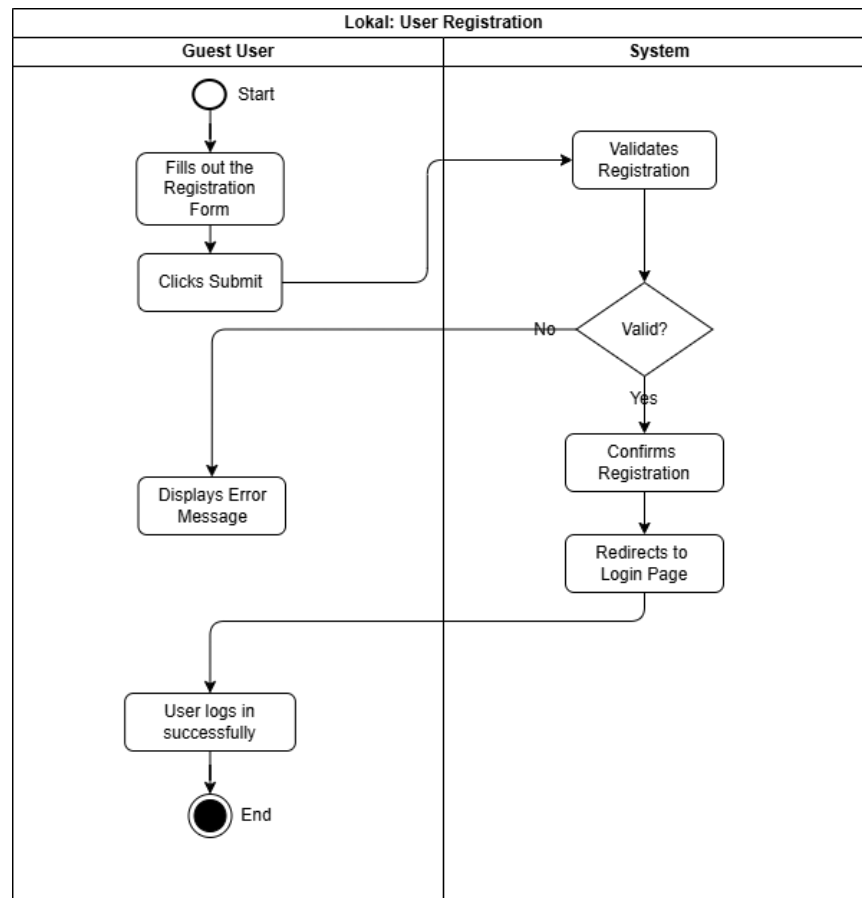


Figure 2. Activity Diagram for User Registration

- Wireframe

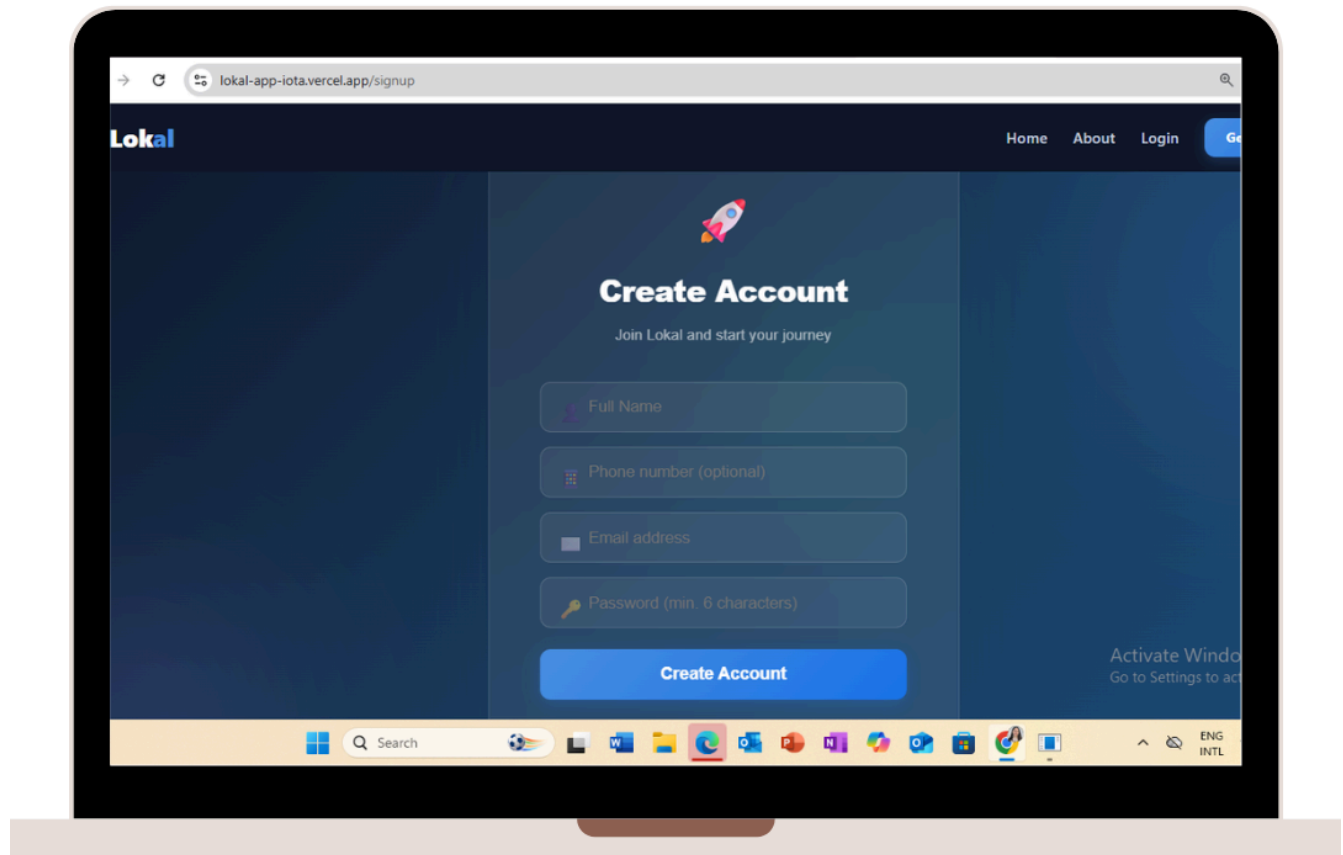


Figure 3. A User Registration Page

1.2 Transaction Name: User Login

- Use Case Diagram

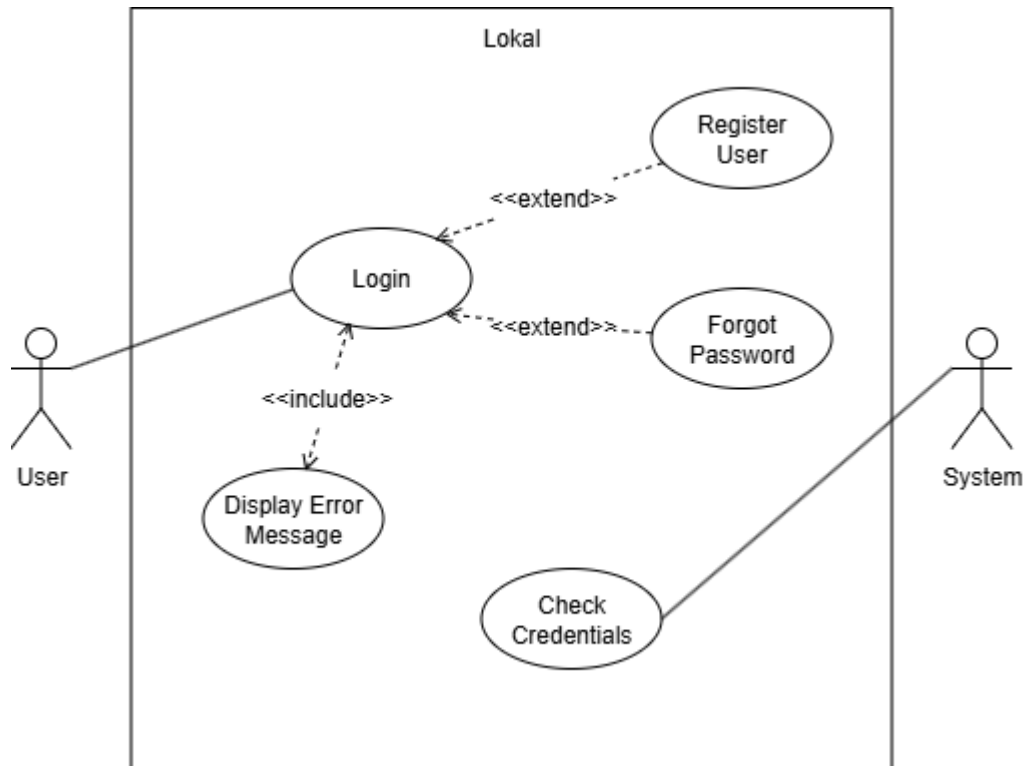


Figure 4. Use Case Diagram for User Login

- **Use Case Description**
 - **Actors:** Registered User
 - **Description:** The registered user enters their email and password to log in to the system. The system validates the credentials and grants access to the appropriate dashboard.
 - **Preconditions:** The user must have a registered account and valid login credential
 - **Postconditions:** The user is successfully logged in and redirected to the dashboard.
- **Main Success Scenario:**
 1. The user opens the login page.
 2. The user enters their email and password.
 3. The system validates the login credentials.
 4. The user is redirected to the dashboard.

• Activity Diagram

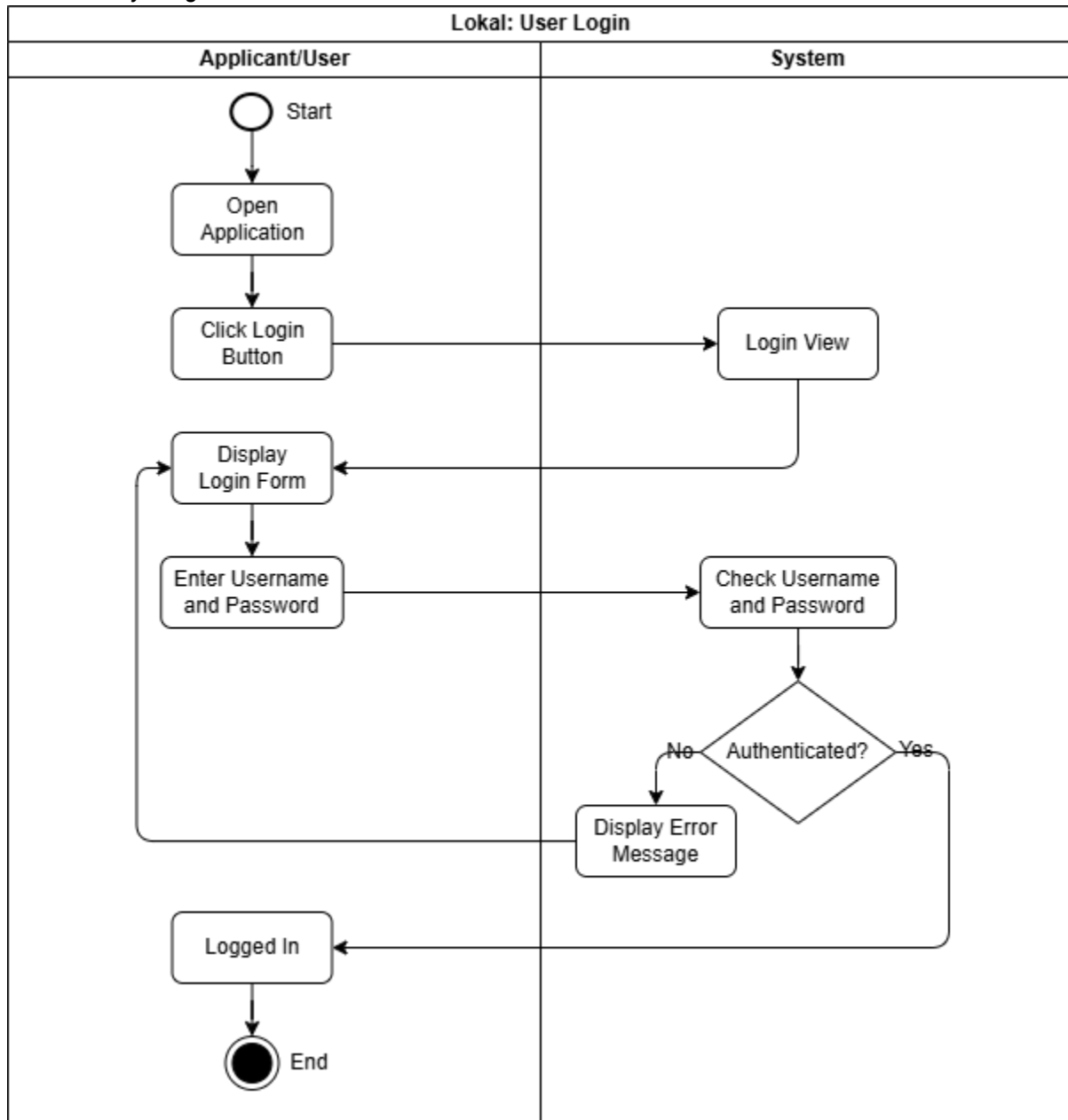


Figure 5. Activity Diagram for User Login

- Wireframe

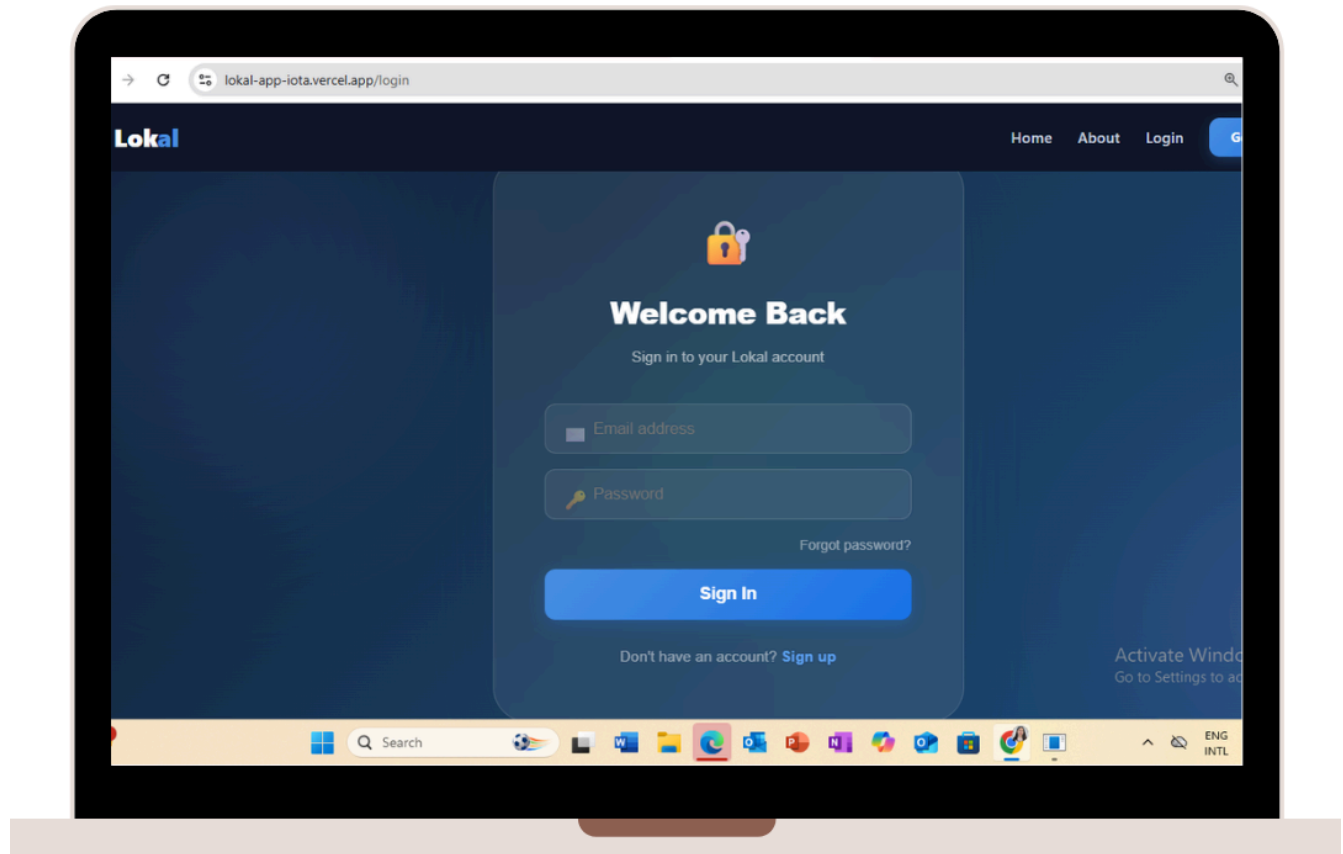


Figure 6. A Login Page of Lokal

- Wireframe

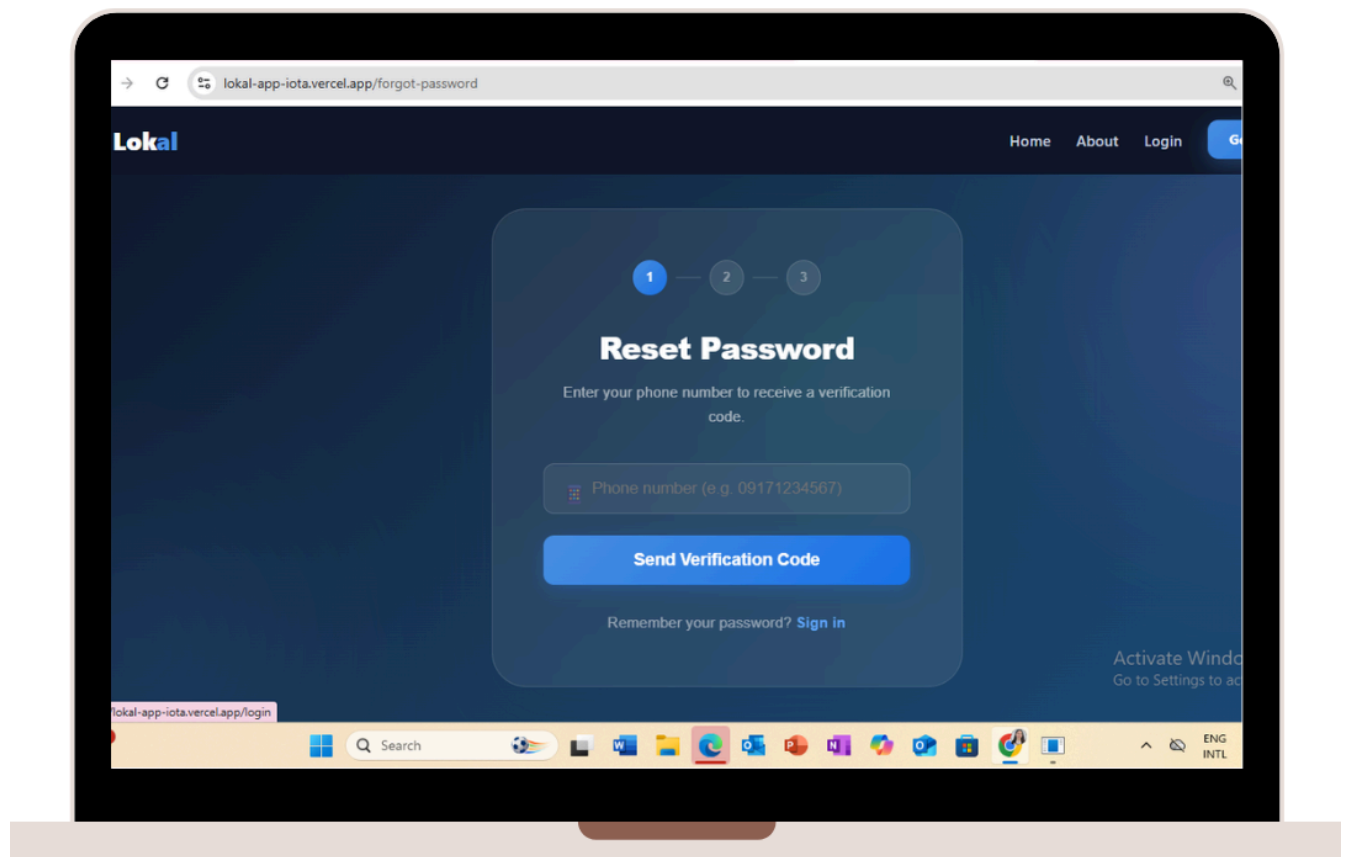


Figure 7. A Reset Password Page of Lokal

Module 2 – Applicant Management

2.1 Transaction Name: Resume/Profile Submission

- Use Case Diagram

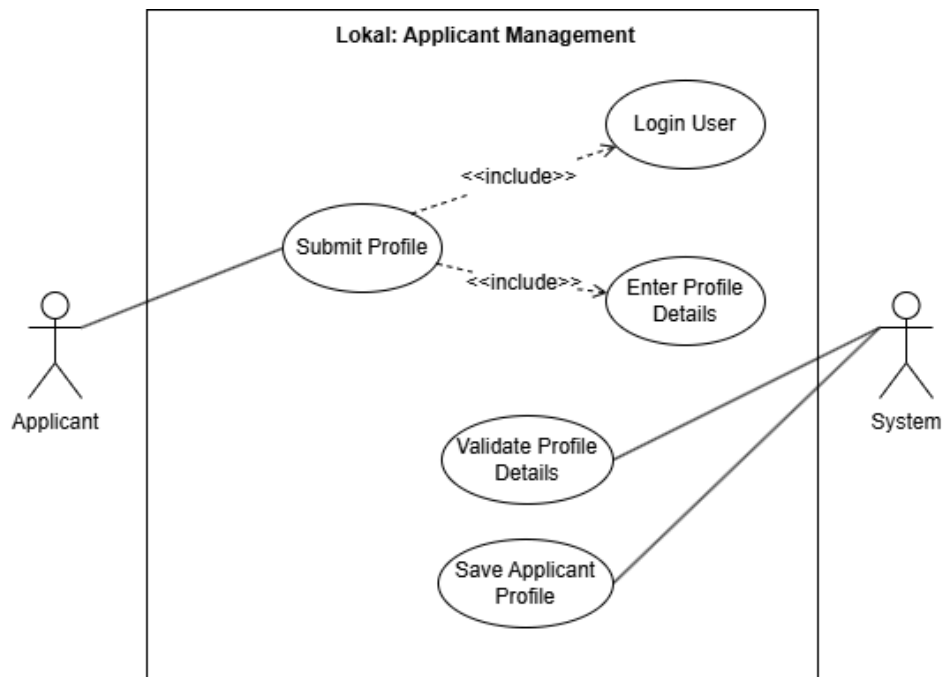


Figure 8. A Use Case Diagram for Resume/Profile Submission

- Use Case Diagram

An applicant submits their profile information, including personal details, skills, work experience, education, resume, and profile photo.

- **Preconditions:** The user must be logged in and have an active applicant account.
- **Postconditions:** The applicant profile is saved and available for AI job matching.

Main Success Scenario:

1. The applicant opens the profile submission form.
2. The applicant enters profile information and uploads the required documents.
3. The system validates the submitted information.
4. The applicant profile is successfully saved.

Extensions: If validation fails, the system prompts the applicant to correct the errors.

- *Activity Diagram*

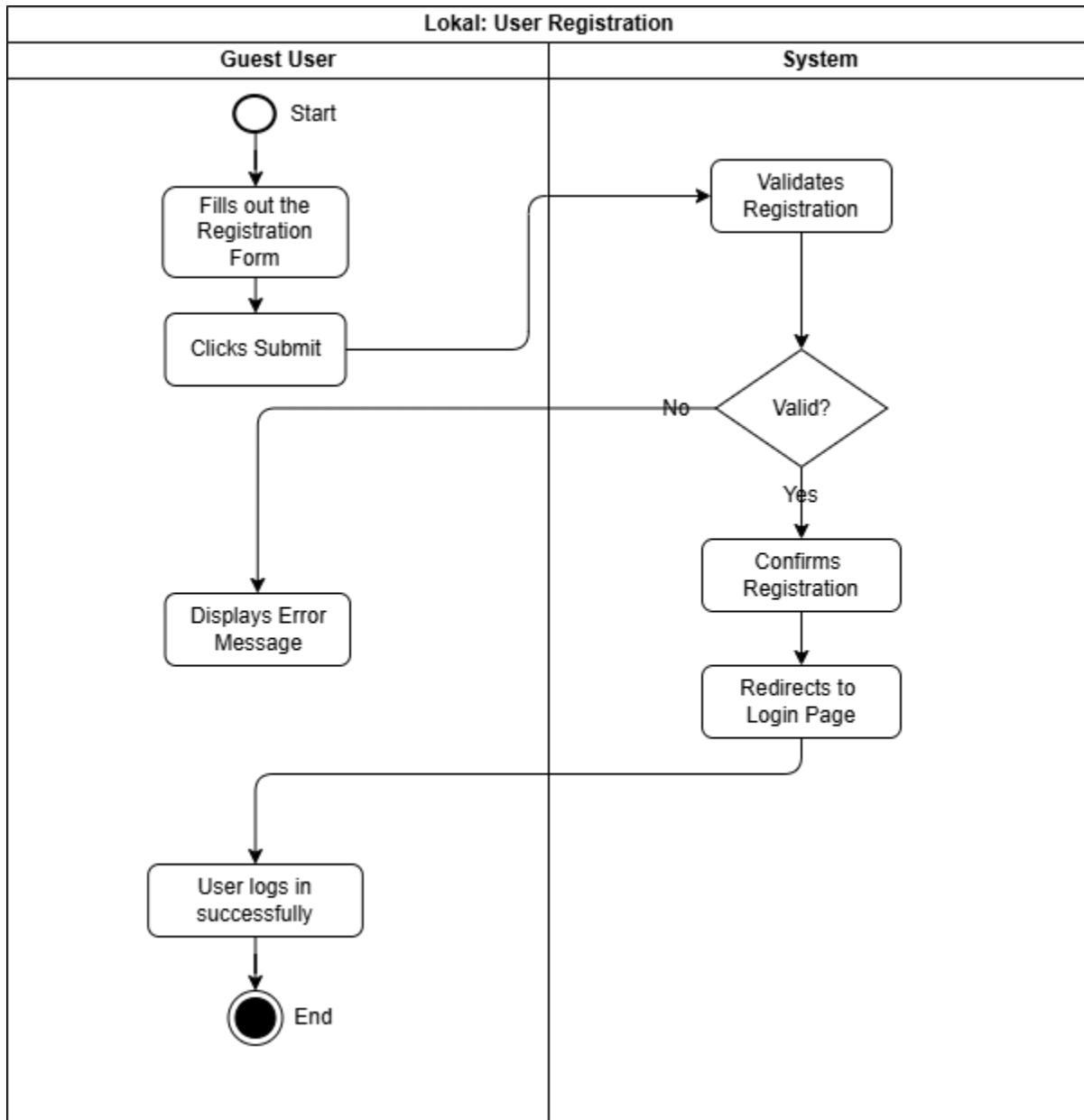


Figure 9. An Activity Diagram for Job Application

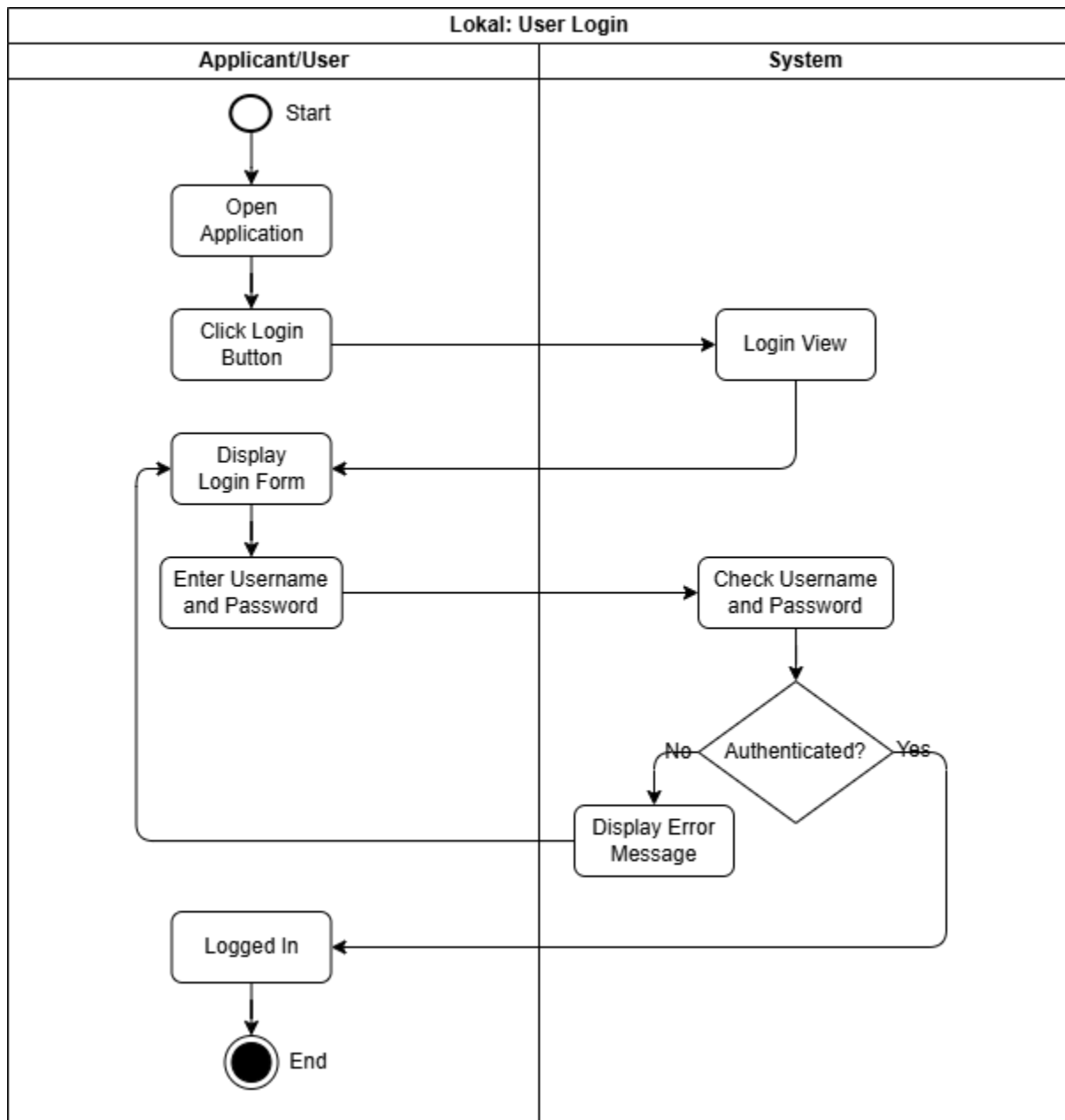


Figure 9. An Activity Diagram for Job Application

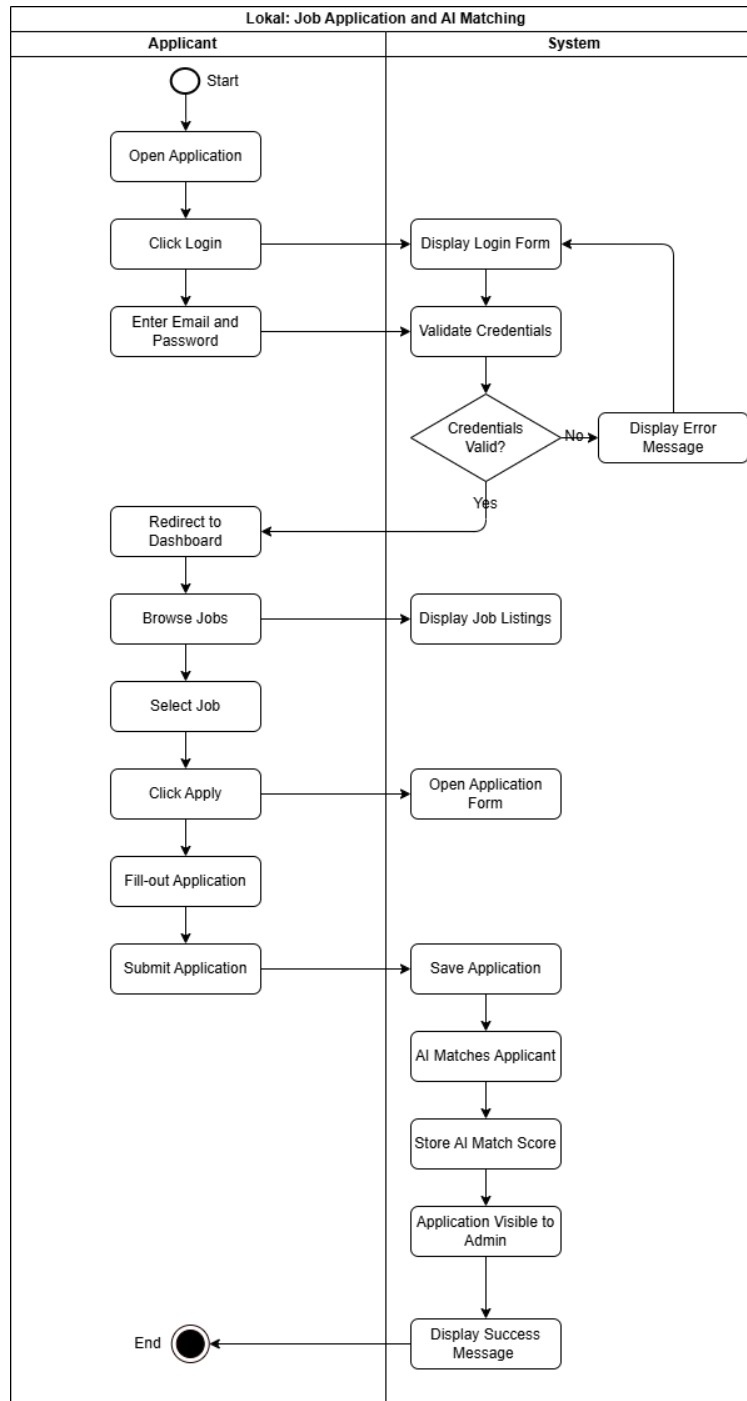


Figure 9. An Activity Diagram for Job Application

- Wireframe

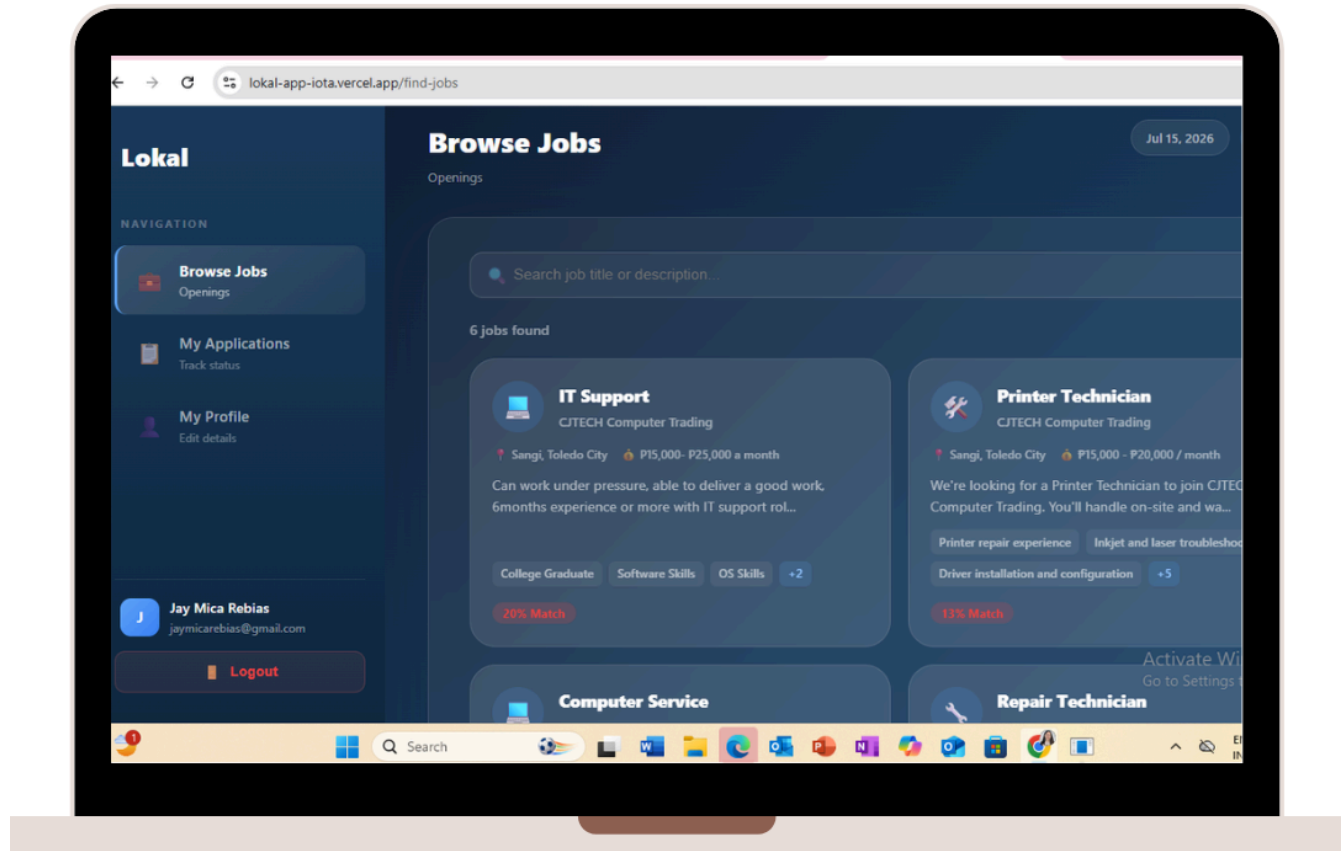


Figure 10. Applicant Browse Jobs Dashboard of LOKAL

2.2 Transaction Name : Applicant Management

- Use Case Diagram

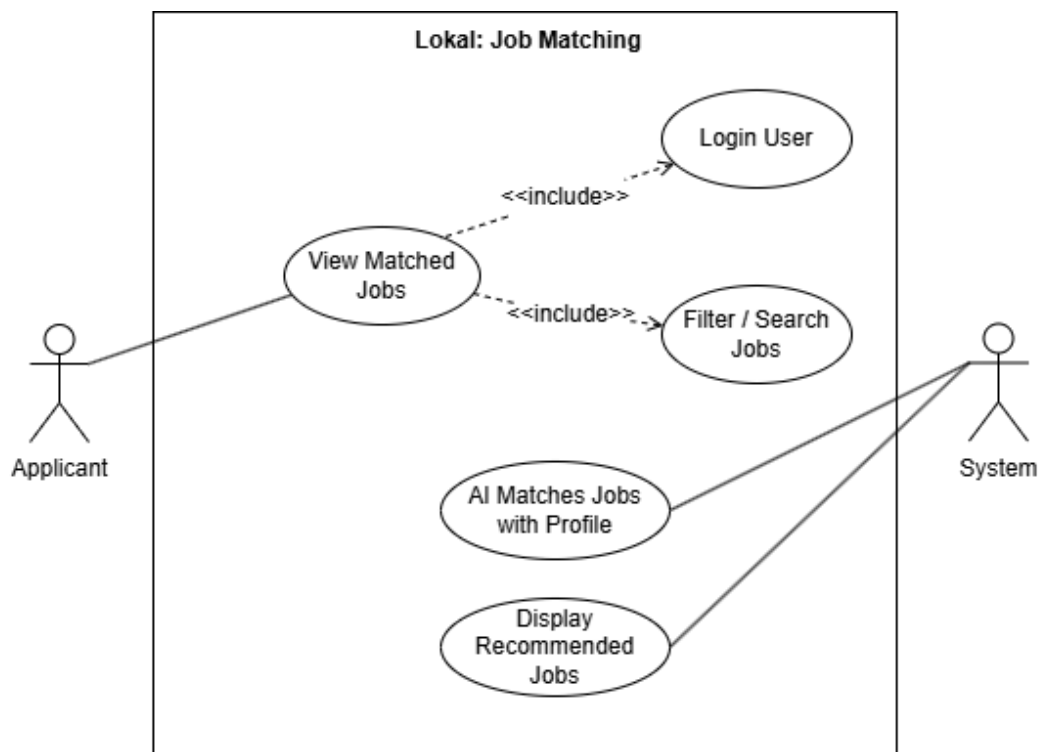


Figure 11. A Use Case Diagram for Job Matching

- Use Case Description

The system analyzes the applicant's profile and recommends job listings that best match their skills, experience, and preferences.

Preconditions: The applicant must be logged in and have a completed profile.

Postconditions: The system displays a list of recommended job matches.

Main Success Scenario:

1. The applicant accesses the job matching page.
2. The system analyzes the applicant's profile.
3. The system displays recommended job listings.
4. The applicant views and filters job matches as needed.
5. The applicant views and filters job matches as needed.

Extensions: If no matching jobs are found, the system shows relevant suggestions or alternative job listings.

- Activity Diagram

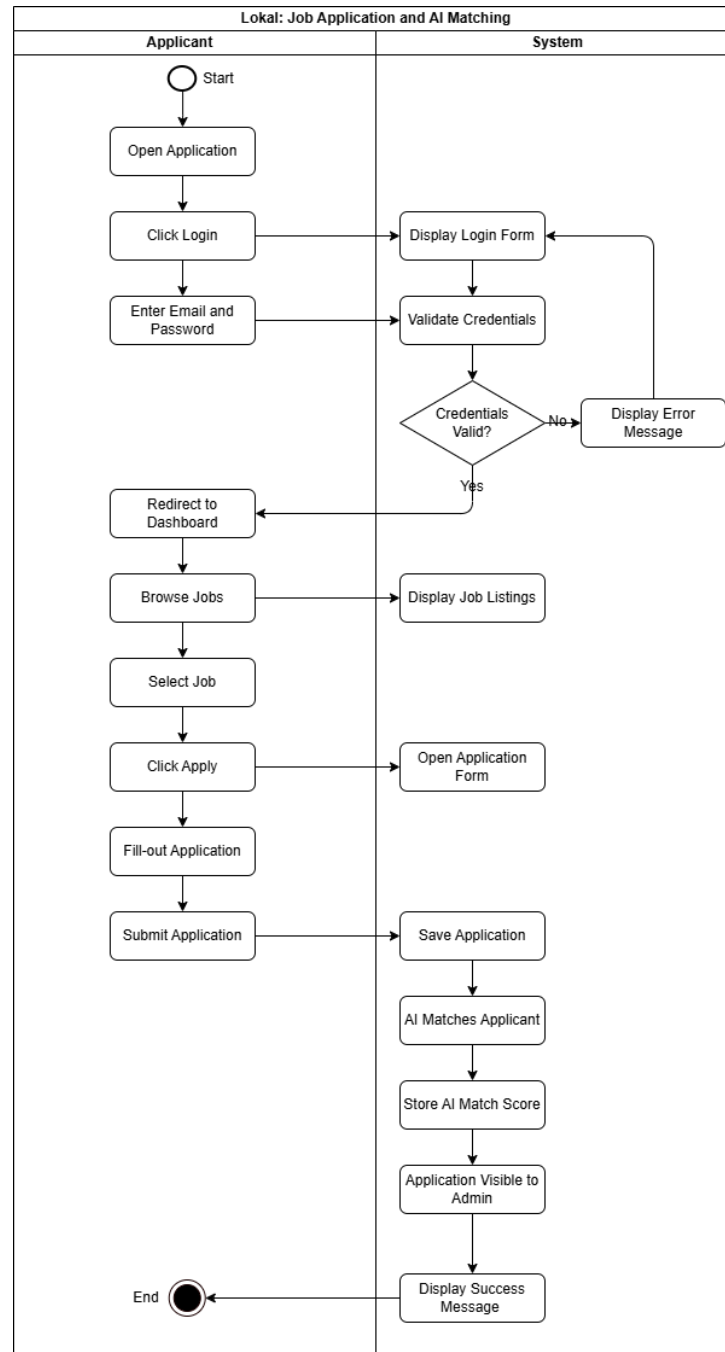


Figure 12. Activity Diagram for Job Application and AI Matching

- Wireframe

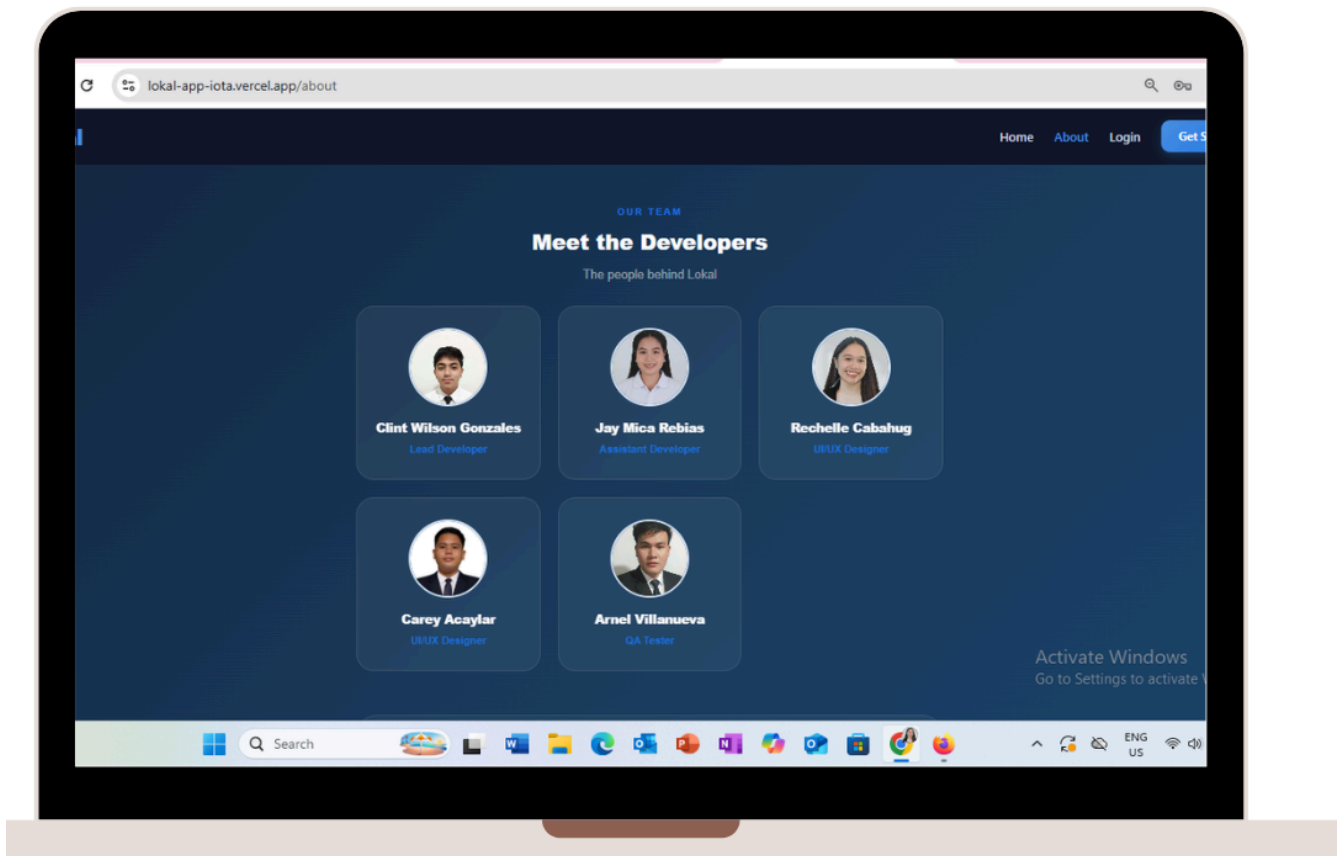


Figure 13. A Developers Page of LOKAL

- Wireframe

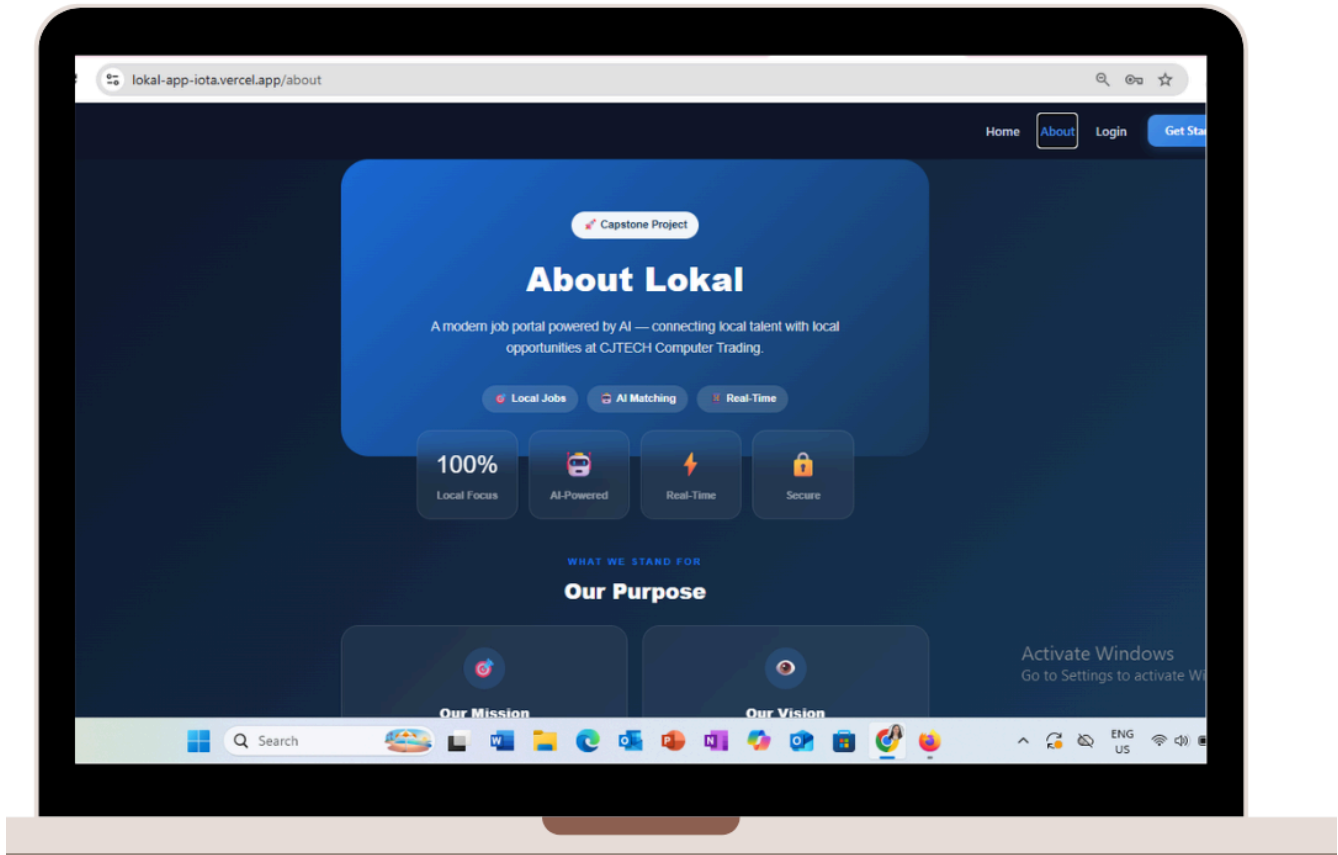


Figure 14. An About Page of LOKAL

3.4 Non-functional requirements

Performance

The **Lokal** must meet the following performance criteria to ensure a smooth user experience for applicants and administrators:

- **Response Time:**
 - The system should load pages within **2 seconds** under normal network conditions.
 - **AI job matching** results should be displayed promptly after profile submission.
- **Concurrent Users:**
 - The system should support **multiple applicants and administrators** accessing the platform simultaneously without significant performance degradation.
- **Database Performance:**
 - Applicant profiles, job postings, and **AI matching results** should be retrieved efficiently from the database.

Security

Security is essential for the **LOKAL** system because it stores applicant information and job postings.

- **User Authentication:**
 - User accounts shall be authenticated using **Supabase Authentication**.
 - Passwords shall be **securely encrypted** before storage.
- **Data Encryption:**
 - All communication between users and the system shall use **HTTPS**.
 - Applicant information shall be protected using **Row-Level Security (RLS) in Supabase**.
- **Access Control:**
 - **Role-based access** shall ensure that applicants and administrators only access authorized features.

Reliability

The **LOKAL** system must provide reliable service to applicants and administrators.

- **Uptime:**
 - The system should remain available with **minimal downtime**.
- **Redundancy:**
 - The web application is hosted entirely on **Supabase's cloud infrastructure**, which provides built-in **automatic backups** and **high availability** across its server clusters.
- **Error Handling:**
 - Invalid inputs and system errors should display **user-friendly error messages**.
- **Database Consistency:**
 - Applicant profiles, applications, and **AI matching results** should be stored consistently and accurately.